Section 07: フルスタック実装

React と Hono を繋ぐ

これまで別々に学んできた「フロントエンド (React)」と「バックエンド (Hono)」を合体させ、1つの「Webアプリケーション」として動かす方法を学びます。

0. 復習：フロントとバックの役割分担

Webアプリは、フロントエンドとバックエンド間での通信で成り立っています。

	レストランの例	Webアプリの例
フロントエンド	ホールスタッフ（お客様と直接やりとり）	React（画面の表示・ボタン操作の受付）
バックエンド	キッチン（注文を受けて料理を作る）	Hono（データの読み書き・処理）
通信	注文票のやりとり	HTTP リクエスト / レスポンス

フロントエンド (React) の仕事

- 画面を描画する (UI)
- ユーザーの操作（入力・クリック）を受け取る
- バックエンドに **「このデータください！」** とリクエストを送る
- 返ってきた結果を画面に反映する

バックエンド (Hono) の仕事

- フロントからのリクエストを受け取る
- データベース (D1 など) からデータを読み書きする
- 結果を **JSON** というデータ形式で返す

> この 2 つが「リクエスト（お願い）」と「レスポンス（返事）」をやり取りすることで、Web アプリが動いています。

1. CORS：最初の壁

React から Hono へ、単純にリクエストを送ると…ブラウザのコンソールに**赤いエラー**が出ます。

```
Access to fetch at 'http://localhost:8787/api' from origin
'http://localhost:5173' has been blocked by CORS policy.
```

初めて見ると「何も悪いことしてないのに！」と思いますが、これはブラウザが皆さんを守ってくれているからこそ出るエラーです。

なぜエラーになるの？

ブラウザには **「同一オリジンポリシー」** という強力なセキュリティルールがあります。

- **オリジン** = `http://localhost:5173` のような「プロトコル + ドメイン + ポート番号」の組み合わせ
- React は `localhost:5173`、Hono は `localhost:8787` → **ポート番号が違う = 別オリジン**
- ブラウザは「**別オリジンへのリクエストは、許可がない限りブロックする**」というルールを持っている

> たとえると：学校で「他のクラスの教室に勝手に入っちゃダメ」というルールがあるようなものです。入りたいなら、**相手のクラスの先生（= バックエンド）が「この人は OK だよ」と許可を出す**必要があります。

CORS エラーの解決策

バックエンド（Hono）側に **「このオリジンからのアクセスは許可します」** という設定を追加します。この仕組みを **CORS (Cross-Origin Resource Sharing)** と呼びます。

```
import { Hono } from "hono";
import { cors } from "hono/cors";

const app = new Hono();

app.use(
  "/*",
  cors({
    origin: "http://localhost:5173", // React アプリのオリジンを許可
    credentials: true,
  }),
);
```

- `"/*"` … Hono のすべてのルート（API）に対して CORS を有効にする
- `origin: "http://localhost:5173"` … 「このオリジンからのリクエストだけ OK」と宣言
- `credentials: true` … cookie などの認証情報を一緒に送ることも許可

> **注意**: 開発中は `localhost:5173` を許可しますが、本番環境では実際のドメイン名（例：`https://myapp.example.com`）に書き換える必要があります。

2. 実践：ブラウザからデータを取得する (GET)

Hono 側に `/api` というエンドポイントがあるとします。React からそのデータを取得して画面に表示してみましょう。

まず知っておきたい 3 つの道具

道具	何をするもの？	たとえ
`useState`	取得したデータを React の「記憶」に保存する	メモ帳に書き留める
`useEffect`	画面が表示された「最初の 1 回だけ」処理を実行する	「お店が開いたら最初にやること」リスト
`fetch`	ブラウザからバックエンドへ通信する **標準の機能**	電話をかけて情報をもらう

`fetch` って何？

`fetch` は、**ブラウザに最初から入っている「通信するための関数」** です。URL を渡すだけで、その URL にリクエストを送ってくれます。

```
const res = await fetch("http://localhost:8787/api");
```

- `fetch(URL)` → 指定した URL に「データをください」とリクエストを送る
- `await` → 通信が終わるまで **待つ**（通信には時間がかかるため）
- `res` → バックエンドからの **返事（レスポンス）** が入る

非同期処理（`async` / `await`）

通信やファイル読み込みなどの **「時間がかかる処理」** を扱うための書き方です。

- `async` … 「この関数の中では `await` を使うよ」という宣言
- `await` … 「この処理が終わるまで次の行に進まないで待ってね」

```
const fetchMessages = async () => {
  const res = await fetch("http://localhost:8787/api"); // ← 通信完了まで待つ
  const data = await res.json(); // ← JSON変換が終わるまで待つ
};
```

> `await` がないと、通信が終わる前に次の行が実行されてしまい、「データがまだ届いてないのに使おうとしてエラー！」となります。

React 側の実装例 (GET)

```
import { useState, useEffect } from "react";

const App = () => {
  // ① データを保存する State を作る（初期値は空の配列）
  const [messages, setMessages] = useState([]);

  // ② 画面が表示された時に「1回だけ」データを取りに行く
  useEffect(() => {
    const fetchMessages = async () => {
      const res = await fetch("http://localhost:8787/api");
      const data = await res.json();
      setMessages(data);
    };
    fetchMessages();
  }, []); // ← 空配列 = 「画面が最初に表示された時だけ」実行

  // ③ 取得したデータを画面に表示する
  return (
    <div>
      {messages.map((msg) => (
        <p key={msg.id}>{msg.text}</p>
      ))}
    </div>
  );
};
```

処理の流れ:

1. 画面が表示される → `useEffect` が発火
2. `fetch` でバックエンド（Hono）に「データください」とリクエスト
3. バックエンドが JSON でデータを返す
4. `setMessages(data)` で State に保存 → React が画面を再描画 → データが表示される

3. 実践：ブラウザからデータを送信する (POST)

今度は逆方向です。ユーザーが入力した情報を、**フロントエンドからバックエンドへ送信（保存）** してみましょう。

GET と POST の違い

	GET	POST
やること	データを「もらう」	データを「送る」
たとえ	レストランで「メニュー見せて」	レストランで「これ注文します」
データの場所	URL だけで完結	リクエストの「本文（Body）」に入れる

GET は URL を渡すだけで OK でしたが、POST は **「何を送るのか」を一緒に詰めて届ける**必要があります。

POST で `fetch` に渡す 3 つの設定

`fetch` に URL だけでなく、**設定オブジェクト（第 2 引数）** を渡します。

```
await fetch("http://localhost:8787/api", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ text: inputText }),
});
```

設定	意味	たとえ
`method: "POST"`	「データを送りますよ」と伝える	手紙に「送付」のスタンプを押す
`headers: { "Content-Type": "application/json" }`	「JSON 形式で送ります」と伝える	封筒に「中身は日本語です」と書く
`body: JSON.stringify(...)`	実際のデータを JSON 文字列に変換して入れる	封筒の中に手紙を入れる

> `JSON.stringify()` は JavaScript のオブジェクト（`{ text: "こんにちは" }`）を、通信で送れる **文字列**（`'{"text":"こんにちは"}'`）に変換する関数です。

React 側の実装例 (POST)

```
const [inputText, setInputText] = useState("");

const handleSubmit = async (e) => {
  e.preventDefault(); // フォーム送信時のページリロードを防ぐ（※）

  await fetch("http://localhost:8787/api", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ text: inputText }),
  });

  setInputText(""); // 送信後に入力欄をクリア
  // ※ この後に一覧を再取得する処理を入れると画面が最新になります
};
```

> **`e.preventDefault()` とは？**
>
> HTML の `<form>` は、送信ボタンが押されると **ページ全体をリロードする** のがデフォルトの動作です。
> React ではページをリロードせずに通信したいので、この「デフォルト動作」をキャンセルしています。

4. ワークショップ：Part 5 の掲示板 API に React をつなげよう

Part 5 で皆さんが作った **掲示板 API（`GET /messages` / `POST /messages`）** を覚えていますか？
今回は、その API に **React（フロントエンド）を接続** して、ブラウザから投稿・閲覧できるようにします。

今回やること

```
[ブラウザ（React）]  ──GET /messages──→  [Part 5 で作った Hono API]
        ↑                                          │
        └──── JSON でメッセージ一覧 ────────────────┘

[ブラウザ（React）]  ──POST /messages──→  [Part 5 で作った Hono API]
  { name: "太郎", content: "Hello!" }      D1 に保存
        ↑                                          │
        └──── 更新後の一覧を返す ────────────────┘
```

バックエンド（Hono）は **ほぼそのまま**。変更点は **CORS の追加だけ** です。
メインの作業は **React 側の実装** になります。

Q1: Hono に CORS を追加する

Part 5 のバックエンドは、今まで Postman やターミナルからしかアクセスしていませんでした。ブラウザ（React）から接続するには **CORS の許可** が必要です。

- **やること**
  - Part 5 の Hono プロジェクトを開く
  - `hono/cors` から `cors` をインポートする
  - `app.use("/*", cors({ origin: "http://localhost:5173" }))` を追加する
  - Section 1 で学んだコードがそのまま使えます

> これだけで、Part 5 の `GET /messages` と `POST /messages` がブラウザから呼べるようになります。

Q2: React でメッセージ一覧を表示する (GET)

Vite + React プロジェクトを用意し、Part 5 の `GET /messages` からデータを取得して画面に表示しましょう。

- **ヒント**
  - `useState` で `messages` 配列を管理する
  - `useEffect` の中で `fetch("http://localhost:8787/messages")` を呼ぶ
  - `await res.json()` で取得した配列を `setMessages` で State に保存する
  - JSX で `messages.map(...)` を使って一覧表示する
  - Section 2 の GET 実装例がそのまま参考になります（URL とプロパティ名だけ変える）

Q3: React からメッセージを投稿する (POST)

入力欄と送信ボタンを作り、Part 5 の `POST /messages` にデータを送りましょう。

- **ヒント**
  - Part 5 の API は `{ name: "名前", content: "本文" }` という JSON を受け取る
  - `useState` で `name` と `content` の 2 つの入力値を管理する
  - `fetch` の `method: "POST"` / `headers` / `body` は Section 3 で学んだ形と同じ
  - 送信後に `GET /messages` を再度呼んで一覧を更新すると、投稿がすぐ画面に反映される

5. 発展課題：ローディング状態の管理

通信には **「時間」がかかります**（数ミリ秒〜数秒）。その間ユーザーに何もフィードバックがないと、「ボタン押せたかな？もう一回押そうかな？」と不安になり、二重送信の原因にもなります。

やってみよう

`isLoading` という **boolean の State** を追加して、通信中の UX を改善しましょう。

- **考え方**
  i. `const [isLoading, setIsLoading] = useState(false);` を用意する
  ii. `fetch` を呼ぶ **直前** に `setIsLoading(true)` にする
  iii. `fetch` が **完了したら** `setIsLoading(false)` に戻す
  iv. `isLoading` が `true` の間だけ：
    - ボタンのラベルを「送信中...」に変える
    - `<button disabled={isLoading}>` でボタンを押せなくする

> 実際のプロダクトでも、「スピナー（くるくる回るアイコン）」や「ボタンの無効化」はよく使われるテクニックです。小さな工夫ですが、ユーザー体験が格段に良くなります。

まとめ

今日、皆さんは **「フロントエンドから API へリクエストを送り、バックエンドがそれに応答する」** という Web システムの真髄を体験しました。

今日学んだことの整理

学んだこと	ポイント
CORS	ブラウザのセキュリティルール。バックエンド側で「許可」を設定する
GET リクエスト	データを「もらう」通信。`fetch(URL)` だけで OK
POST リクエスト	データを「送る」通信。`method` / `headers` / `body` を設定する
async / await	通信のような「時間がかかる処理」を待つための書き方
JSON	フロントとバックが会話するときの「共通言語」

全体像のおさらい

```
[React]                                  [Hono]
   │                                        │
   │── GET /api ──────────────────────────→ │   「データください」
   │← JSON レスポンス ────────────────────│   「はいどうぞ」
   │                                        │
   │── POST /api (JSON body) ──────────→ │   「これ保存して」
   │← 更新後の JSON ──────────────────────│   「保存したよ、最新一覧はこれ」
```

これが **フルスタック開発の基本形** です。ここから先は、データベースを繋いだり、認証を追加したり、AI を組み込んだりすることができますが、**通信の仕組み自体は今日学んだことと同じ**です！